```
import pandas as pd

url = 'https://gist.githubusercontent.com/cloudwalk-tests/76993838e65d7e0f988f40f1b1909c97/raw/295d9f7cb8fdf08f3cb3bdf
1696ab245d5b5c1c9/transactional-sample.csv'

df = pd.read_csv(url)

df['transaction_date'] = pd.to_datetime(df['transaction_date']) # making the transaction_date a proper type

"""
Data columns (total 8 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   transaction_id      3199 non-null   int64
 1   merchant_id         3199 non-null   int64
 2   user_id             3199 non-null   int64
 3   card_number         3199 non-null   str
 4   transaction_date    3199 non-null   datetime64[us]
 5   transaction_amount  3199 non-null   float64
 6   device_id           2369 non-null   float64
 7   has_cbk             3199 non-null   bool
dtypes: bool(1), datetime64[us](1), float64(2), int64(3), str(1)
"""
'''
transaction_id  merchant_id  user_id       card_number           transaction_date  transaction_amount  device_id  has_
cbk
0        21320398        29744    97051  434505******9116 2019-12-01 23:16:32.812632              374.56   285475.0
 False
1        21320399        92895     2708  444456******4210 2019-12-01 22:45:37.873639              734.87   497105.0
  True
2        21320400        47759    14777  425850******7024 2019-12-01 22:22:43.021495              760.36        NaN
 False
3        21320401        68657    69758  464296******3991 2019-12-01 21:59:19.797129             2556.13        NaN
  True
4        21320402        54075    64367  650487******6116 2019-12-01 21:30:53.347051               55.36   860232.0
 False
'''

# ============================================================
# 0. DATA PREPARATION
# ============================================================

# Let's oder by user_id so we can check if the same user is testing different cards
df_sorted = df.sort_values(by=['user_id', 'transaction_date'])


# ================================================================
# 1. VELOCITY ANALYSIS (time between transactions of the same user)
# ================================================================

# Checking the time difference between transactions for the same user
df_sorted['time_delta'] = df_sorted.groupby('user_id')['transaction_date'].diff()
```

```
# checking a specific and suspicious user
user_266_data = df_sorted[df_sorted['user_id'] == 266]
'''
user_id           transaction_date             time_delta
3124      266 2019-11-03 20:24:50.039652                    NaT
3123      266 2019-11-03 20:25:23.212894 0 days 00:00:33.173242
'''

# Converting the time difference into total seconds
df_sorted['seconds_since_last'] = df_sorted['time_delta'].dt.total_seconds()

# Let's filter the dataset to only keep transactions that happened within 60 seconds of a previous one
fast_transactions = df_sorted[df_sorted['seconds_since_last'] < 60]

'''
Chargbacks for transactions under 60 seconds:
has_cbk
False    9
True     9
Name: count, dtype: int64
'''

# checking for transactions between 60 and 120 seconds
medium_fast_transactions = df_sorted[(df_sorted['seconds_since_last'] >= 60) & (df_sorted['seconds_since_last'] <= 120
)]

'''
Chargebacks for transactions between 60 and 120 seconds:
has_cbk
False    11
True      9
Name: count, dtype: int64
'''


# Let's look at the user, device and cards used for these impossibly fast transactions
'''
Inspecting the bots (fast transactions):
      user_id  device_id       card_number  has_cbk
3123      266        NaN  482425******1320    False
964     10378    17372.0  415944******1540     True
2573    10405   856642.0  515894******4290    False
3102    16781        NaN  546056******2924     True
2943    42677        NaN  515601******8618     True
2791    49106        NaN  651660******3628    False
1103    53850    20098.0  527468******1757     True
337     56877   866529.0  406655******8217     True
1484    67903   321795.0  478308******3072    False
2931    75710        NaN  554482******7640     True
2929    75710        NaN  554482******7640     True
2355    76708   760553.0  544731******4582    False
```

```
3140    76819        NaN  552289******8870     True
130     77959   589318.0  432957******7262    False
129     77959   589318.0  432957******7262    False
628     88553    27250.0  410863******7755    False
147     90176   705388.0  545368******9514     True
1411    98739   422057.0  606282******2118    False

Here we have fast operations. Looks like users 75710 and 77959 are trying to extract money before the bank's automated
 system locks the card.
device_id missing (NaN) is a strong signal that an API is being used, because automated scripts don't generate device
fingerprints like a regular web browsers or mobile phones do.
'''
# ============================================================
# 2. MISSING DEVICE_ID (GHOST DEVICES)
# ============================================================

# Filtering the entire dataset for rows where device_id is missing (NaN)
missing_device_data = df_sorted[df_sorted['device_id'].isna()]

'''
Chargebacks for missing device ids (ghosts devices):
has_cbk
False    763
True      67
Name: count, dtype: int64
'''

# ============================================================
# 3. HIGH AMOUNT ANALYSIS
# ============================================================

# 3.1 Global high amounts: checking transactions in the top 5% of all spending
high_amount_threshold = df_sorted['transaction_amount'].quantile(0.95)
global_high_amounts = df_sorted[df_sorted['transaction_amount'] > high_amount_threshold]

'''
Chargebacks for Global High Amounts (Top 5% over 2775.27):
has_cbk
False    98
True     62
Name: count, dtype: int64

# here we have around 38% fraud rate
# Looks like when a fraudsters get a working card, they try to cash out massive amounts before the bank catches on.
'''

# 3.2 Behavioral Spikes: Checking transactions that are 3x larger than the user's personal average
# Using .transform('mean') neatly to assign the user's average back to each of their transaction rows
df_sorted['user_avg_amount'] = df_sorted.groupby('user_id')['transaction_amount'].transform('mean')

# Flaging transactions that spike 3x higher than the user's norm
df_sorted['is_spike'] = df_sorted['transaction_amount'] > (df_sorted['user_avg_amount'] * 3)
```

```
user_spikes = df_sorted[df_sorted['is_spike']]

'''
Chargebacks for Sudden Spikes (3x higher than user average):
has_cbk
True     1
False    1
Name: count, dtype: int64

''' # most users in this dataset only have 1 or 2 transacions on record; for those who have only one recorded transact
ion, it is impossible to have another that is 3x higher.



# ============================================================
# 4. COMBINED RISK SIGNALS
# ============================================================

# Combining signals: Missing Device ID and Amount over 2000
combined_risk = df_sorted[(df_sorted['device_id'].isna()) & (df_sorted['transaction_amount'] > 2000)]

print("\nChargebacks for Ghosts making Large Purchases (>2000):")
print(combined_risk['has_cbk'].value_counts())
'''
Chargebacks for Ghosts making Large Purchases (>2000):
has_cbk
False    90
True     27
Name: count, dtype: int64
''' # fraud rate here: about 23%. The problem here is: while we stopt 27 fraudsters we would, also, block 90 good cust
momers who are trying to spend over $2,000 each one!

# ============================================================
# 5. TOWARDS A PRACTICAL APPROACH â\200\223 RISK SCORING
# ============================================================

# We are going to use a Risk Scoring to block only those transactions, that cross a very specific point threshold. The
se blocked transactions may being analised manually.

df_sorted['risk_score'] = 0

# role nÂ°1: 100 points for velocity (under 60 secondsd)
is_fast = df_sorted['seconds_since_last'] < 60
df_sorted.loc[is_fast, 'risk_score'] += 100

# role nÂ° 2: 30 points for missing Device ID
is_ghost = df_sorted['device_id'].isna()
df_sorted.loc[is_ghost, 'risk_score'] += 30

# role nÂ° 3: 20 points for global high amount
is_high_value = df_sorted['transaction_amount'] > high_amount_threshold
df_sorted.loc[is_high_value, 'risk_score'] += 20
```

```
# sorting the dataset by the highest risk score to see our most dangerous transactions
high_risk_transactions = df_sorted.sort_values(by='risk_score', ascending=False)

#print("\nTop 10 Highest Risk Transactions:")
#print(high_risk_transactions[['user_id', 'transaction_amount', 'risk_score', 'has_cbk']].head(10))
'''
Top 10 Highest Risk Transactions:
      user_id  transaction_amount  risk_score  has_cbk
2791    49106             4043.43         150    False
2943    42677              301.58         130     True
3140    76819             1038.47         130     True
3102    16781              502.16         130     True
3123      266              235.70         130    False
2931    75710              254.37         130     True
2929    75710              599.13         130     True
1484    67903              300.86         100    False
147     90176              582.63         100     True
964     10378              553.66         100     True
(.venv) filipe@data-analytics:~/Documents/cloud_walk$
'''
# ====================================================================
# 6. Exporting to CSV
# ====================================================================

# filtering transactions that hits the threshold (Score >= 100)
flagged_transactions = df_sorted[df_sorted['risk_score'] >= 100]

# Exporting to a CSV file (index=False prevents pandas from writing the row numbers)
flagged_transactions.to_csv('high_risk_alerts.csv', index=False)

#print(f"\nSuccess! Exported {len(flagged_transactions)} high-risk transactions to 'high_risk_alerts.csv'")


# ============================================================
# CONCLUSION
# ============================================================

'''
After analyzing the data, we can see some clear patterns of suspicious behavior:

- Very fast transactions (under 60-120 seconds) from the same user
- Missing device_id (ghost devices), that looks a lot like automated scripts
- High amounts, especially when combined with missing device

The combined signal (missing device + amount over 2000) already shows a fraud rate around 23%. The problem is that if
we just block all of them, we also stop many good customers.

Because of this, the best way is not a simple rule, but a Risk Scoring system. We give points for each risk signal and
 only block or send to manual review the transactions that pass a certain threshold. This way we can catch more frauds
ters without hurting too much the good users.
```

This is just the first analysis. With more data (like IP, location, time of the day, merchant category, etc) the scoring can become much better.

name: Filipe Alves Vieira
'''